Ex 8: Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

Learning Objective:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. **Import Required Libraries**
   Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).
2. **Load Dataset**
   Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.
3. **Data Preprocessing** ○ Clean and normalize text ○ Prepare input-output pairs (query → response)
4. **Split Dataset**
   Divide into training and testing datasets.
5. **Load Pre-trained Model**
   Load models such as T5 or BART from Hugging Face.
6. **Fine-Tune the Model**
   Train the model using the training dataset.
7. **Save the Fine-Tuned Model** Store trained model and tokenizer.
8. **Build Inference Pipeline**
   Create a function to generate responses for input queries.
9. **Generate Predictions on Test Dataset** ○ Pass all test queries to the model ○ Generate responses for each query
10. **Compute BLEU Scores (BLEU-1 to BLEU-4)**

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. **Compute ROUGE Scores**

- ROUGE-1 → unigram recall
- ROUGE-2 → bigram recall

- ROUGE-L for sequence similarity

12. **Aggregate Evaluation Metrics**

- Calculate average scores across all test samples
- Prepare overall performance summary

13. **Display Evaluation Results**

Metric	Score
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. **Develop User Interface**

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. **Deploy the Application**

- Host the model for real-time customer query handling

16. **Test with Sample Queries**
    Validate the deployed system with real-time user inputs.

17. **Display Output**
    Show generated responses in the interface.

Program:

```
# ==========================================================

# IMPORT LIBRARIES
```

```
# ============================================================
import pandas as pd
import numpy as np
from datasets import load_dataset, Dataset
from sklearn.model_selection import train_test_split

from transformers import (
    T5Tokenizer,
    T5ForConditionalGeneration,
    Trainer,
    TrainingArguments
)

# ============================================================
# LOAD DATASET FROM INTERNET
# ============================================================
dataset = load_dataset(
    "bitext/Bitext-customer-support-llm-chatbot-training-dataset"
)

print(dataset)

# ============================================================
# CHECK COLUMN NAMES
# ============================================================
train_raw = dataset["train"].to_pandas()
```

```
print("Columns:")
print(train_raw.columns)

possible_question_cols = [
    "instruction",
    "prompt",
    "question",
    "input",
    "text"
]

possible_answer_cols = [
    "response",
    "output",
    "answer",
    "label"
]

question_col = None
answer_col = None

for col in possible_question_cols:
    if col in train_raw.columns:
        question_col = col
        break

for col in possible_answer_cols:
```

```
    if col in train_raw.columns:
        answer_col = col
        break

print("Question Column:", question_col)
print("Answer Column  :", answer_col)

# ============================================================
# CREATE DATAFRAME
# ============================================================
df = train_raw[[question_col, answer_col]].copy()

df.columns = ["question", "answer"]

# Remove null values
df.dropna(inplace=True)

# Convert to string
df["question"] = df["question"].astype(str)
df["answer"]   = df["answer"].astype(str)

# Use first 3000 rows for faster training
df = df.head(3000)

print(df.head())

# ============================================================
```

```
# TRAIN TEST SPLIT
# ============================================================
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42
)

train_ds = Dataset.from_pandas(train_df)
test_ds  = Dataset.from_pandas(test_df)

print("Train Size:", len(train_df))
print("Test Size :", len(test_df))

# ============================================================
# LOAD MODEL
# ============================================================
model_name = "google/flan-t5-base"

tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

# ============================================================
# TOKENIZATION
# ============================================================
def preprocess(example):
```

```
    inputs = tokenizer(
        "question: " + example["question"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

    targets = tokenizer(
        example["answer"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

    inputs["labels"] = targets["input_ids"]

    return inputs

train_ds = train_ds.map(preprocess)
test_ds  = test_ds.map(preprocess)

# ============================================================
# TRAINING ARGUMENTS
# ============================================================
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=5,
```

```
    per_device_train_batch_size=4,
    learning_rate=2e-5,
    logging_steps=50,
    save_strategy="epoch",
    fp16=True
)

# ============================================================
# TRAIN MODEL
# ============================================================
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds
)

trainer.train()

# ============================================================
# SAVE MODEL
# ============================================================
model.save_pretrained("customer_model")
tokenizer.save_pretrained("customer_model")

print("====================================")
print("MODEL SAVED SUCCESSFULLY")
print("Folder: customer_model")
```

```
print("====================================")
import torch
import evaluate
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

# ============================================================
# LOAD SAVED MODEL
# ============================================================
from transformers import T5Tokenizer, T5ForConditionalGeneration

model_path = "customer_model"

tokenizer = T5Tokenizer.from_pretrained(model_path)
model = T5ForConditionalGeneration.from_pretrained(model_path)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

# ============================================================
# STEP 8: BUILD INFERENCE PIPELINE
# ============================================================
def generate_response(query):

    text = "question: " + query

    inputs = tokenizer(
        text,
```

```
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=64
    )

    input_ids = inputs.input_ids.to(device)
    attention_mask = inputs.attention_mask.to(device)

    output = model.generate(
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_length=30,
        num_beams=5,
        early_stopping=True
    )

    prediction = tokenizer.decode(
        output[0],
        skip_special_tokens=True
    )

    return prediction

# ============================================================
# STEP 9: GENERATE PREDICTIONS ON TEST DATASET
# test_df must already exist from train_test_split
```

```
# ============================================================
predictions = []

for q in test_df["question"]:
    pred = generate_response(q)
    predictions.append(pred)

# ============================================================
# SAVE OUTPUT TO EXCEL
# ============================================================
result_df = pd.DataFrame({
    "Question": test_df["question"].values,
    "Actual_Output": test_df["answer"].values,
    "Predicted_Output": predictions
})

result_df.to_excel("predictions.xlsx", index=False)
result_df.to_csv("predictions.csv", index=False)

print("Saved predictions.xlsx")
print("Saved predictions.csv")

# ============================================================
# STEP 10: COMPUTE BLEU-1 TO BLEU-4
# ============================================================
smooth = SmoothingFunction().method1
```

```
bleu1_scores = []
bleu2_scores = []
bleu3_scores = []
bleu4_scores = []

for actual, pred in zip(result_df["Actual_Output"], result_df["Predicted_Output"]):

    ref = [actual.split()]
    hyp = pred.split()

    bleu1_scores.append(
        sentence_bleu(ref, hyp,
                      weights=(1,0,0,0),
                      smoothing_function=smooth)
    )

    bleu2_scores.append(
        sentence_bleu(ref, hyp,
                      weights=(0.5,0.5,0,0),
                      smoothing_function=smooth)
    )

    bleu3_scores.append(
        sentence_bleu(ref, hyp,
                      weights=(0.33,0.33,0.33,0),
                      smoothing_function=smooth)
    )
```

```
    bleu4_scores.append(
        sentence_bleu(ref, hyp,
                      weights=(0.25,0.25,0.25,0.25),
                      smoothing_function=smooth)
    )

# ============================================================
# STEP 11: COMPUTE ROUGE
# ============================================================
rouge = evaluate.load("rouge")

rouge_result = rouge.compute(
    predictions=result_df["Predicted_Output"].tolist(),
    references=result_df["Actual_Output"].tolist()
)

# ============================================================
# STEP 12: AGGREGATE METRICS
# ============================================================
final_scores = {
    "BLEU-1": np.mean(bleu1_scores),
    "BLEU-2": np.mean(bleu2_scores),
    "BLEU-3": np.mean(bleu3_scores),
    "BLEU-4": np.mean(bleu4_scores),
    "ROUGE-1": rouge_result["rouge1"],
    "ROUGE-2": rouge_result["rouge2"],
```

```
    "ROUGE-L": rouge_result["rougeL"]
}

# ============================================================
# SAVE METRIC SCORES TO EXCEL
# ============================================================
metric_df = pd.DataFrame(
    list(final_scores.items()),
    columns=["Metric", "Score"]
)

metric_df.to_excel("metric_scores.xlsx", index=False)

# ============================================================
# STEP 13: DISPLAY RESULTS
# ============================================================
print("====================================")
print("EVALUATION RESULTS")
print("====================================")
print(metric_df)

print("\nSaved metric_scores.xlsx")
```

Output:

```
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 282/282 [00:00<00:00, 633.56it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them. You sho
Saved predictions.xlsx
Saved predictions.csv
==================================
EVALUATION RESULTS
==================================
    Metric     Score
0   BLEU-1  0.003957
1   BLEU-2  0.001147
2   BLEU-3  0.000653
3   BLEU-4  0.000473
4  ROUGE-1  0.081245
5  ROUGE-2  0.020965
6  ROUGE-L  0.069875

Saved metric_scores.xlsx
```

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.